Proyecto: UNO Game

Reporte de trabajo.



Interfaces Graficas por aplicación 11-12 h.

Morales de la Rosa Edgar Alexis

Castañon Chiquito Iván Alejandro

Arredondo Dominguez Fernando

Reporte de trabajo: Proyecto UNO Game, Segunda Entrega

08 de noviembre de 2026.

Descripción general del proyecto

El proyecto para este parcial consiste en crear el juego de UNO, juego popular basado en el uso de cartas para poder jugar, en cada turno el jugador tiene la libertad de usar cualquier carta siempre y cuando siga con la secuencia de colores, o tenga el mismo número. El jugador también puede usar cartas especiales como, por ejemplo, comodines, para cambiar de color, o la secuencia de turnos, o incluso hacer que el rival tenga que comer cartas. El primer jugador sin quedarse sin cartas gana.

Para ello es necesario usar el lenguaje de programación C# para así lograr crear los eventos necesarios para ejecutar cada mecánica del mismo juego. A su vez, es necesario contar con una base de datos, ya que, en ella se guardará el historial del juego, movimientos, ganadores, partidas, fechas de inicio, tiempo de cada partida, etc.

Segunda entrega.

El proyecto cumple lo estipulado en la primera entrega, pero agrega nuevas modalidades, ahora es un juego web local. Las maquinas interactúan entre si mediante una red local por medio de un servidor que administra todos los movimientos para poder tener un juego “online” localmente.

División de tareas entre los integrantes del equipo

En este caso nuestro equipo solo se compone de dos personas:

Iván Castañón fue el encargado de:

- Dentro de la FormHistorial creó 3 DataGridViews para poder mostrar la información almacenada en la base de datos, el primero pertenece a los datos del jugador, el segundo a los movimientos de los jugadores en esa

partida y el último a los jugadores que aparecieron en ella junto con su estado final de ganadores o perdedores, se muestran los datos tal como aparecen en diagrama entidad relación.

Commit: b5a76ad

- Creó la base de datos, las tablas y los atributos de los datos.

  Commit: ca00609 y 2f134fe

- Creó una nueva pestaña o apartado para poder llevar a cabo la lógica de la partida y también el apartado del historial.

  Commit: 697f3d
- Creó el API junto con las sentencias SQL para poder crear o actualizar los datos de cada partida.

  Commit: 6e89670, 533cef6 y 11cf36f
- Crear las clases del modelo pydantic, que permiten manejar los parámetros recibidos por json para convertirlos a los tipos de datos esperados en python y así crear una sentencia en SQL a la base de datos.

  Commit: 6e89670, b5a76ad y f13b76d
- Dentro de C# agregó un elemento clase en el cual se crearon clases para poder convertir los gets de JSON de la URL en objetos de esas clases y mostrarlos en los DataGridView mientras el programa está en ejecución.

  Commit: 6e89670, b5a76ad y f13b76d
- Al comenzar, durante cada movimiento de los jugadores y al finalizar el juego, creo las cadenas JSON y las envío directamente a la API en Python para almacenar toda la información en la base de datos.

  Commit: 6e89670, b5a76ad y f13b76d
- A su vez dentro de C# agrego un nuevo elemento de clase llamado PartidaAPI, donde se extendieron la cantidad de funciones dentro de la

clase FormPartida (la ventana donde se lleva a cabo la partida), permitiéndole interactuar directamente con la API creada en python por medio del URL.

Commit: 6e89670, b5a76ad y f13b76d

- Correcciones en la implementación de Sockets y El 3er Jugador

  Commit: **8c64a60**

Alexis Morales, por otro lado, fue el encargado de:

- Diseñar la lógica del juego junto con sus consideraciones oficiales (reglas oficiales del juego)

  Commit: sin commit específico, podrían ser los mismos de la parte de funciones y clases del juego.
- Diseñar el apartado visual en las áreas de FormPartida y FormEligeColor, al igual que todas sus implicaciones como elementos (botones, cartas y demás apartados estéticos).

  Commits: 7fa1929, 1d6f368, 757fc3a, dab3290, 03f2644, 81f79af, 0b5b5bc, b8eec2d 5062286 y 0fb43a4.
- Funciones y clases del juego (Mazo.cs y Carta.cs) como los métodos con los que se inicia el mazo, se barajea o se reparte. La lógica de movimientos y restricciones dentro del juego (tomar carta del mazo, jugar carta o comodines, turnos dentro del juego, entre otros.

  Commit: f66fd9f, e42626a, dcbd97d, 9c2e41f, 161f538, 0eaf483, f3b18c2
- Correr las pruebas una vez que Iván Castañón añadía bloques de código que incorporaran la conexión de la base de datos e informar sobre detalles y errores que debían ser corregidos del lado de la base de datos.

  Commit: 11cf36f, 418371c, 2eed8c3.
- Mejora e Implementacion

Commit: 347f37c, 65a2aa4, fad1aad, e7fb958, f2127fc

Fernando Arredondo aporto:

- La Implementación e idea del uso de una sala para dar una sensación más propia a la de un juego online.
  Commit: 77dc7ba, 2a5b6cc.

- Actualización de Form1
  Commit: 398adee,

Ambos nos encargaremos de crear el código, eventos y funciones para que el juego pueda ejecutarse correctamente, es decir, el código fuente será creado entre ambos y una vez que este sea funcional, cada uno se encargará de enfocarse y conectar cada una de sus áreas de trabajo.

Lo mencionado anteriormente fue modificado durante la marcha, al momento de comenzar con las actividades, aligeramos la carga al distribuirnos el entorno de trabajo: Iván gestionaría la base de datos y sus componentes correspondientes (base de datos en SQL, API, servidor Uvicorn y las conexiones dentro del código del juego en Visual Studio) mientras que Alexis se encargaría del juego base y sus eventos, aplanando el terreno para que iban integrará lo correspondiente para hacer el ensamblado a la base de datos.

Segunda entrega.

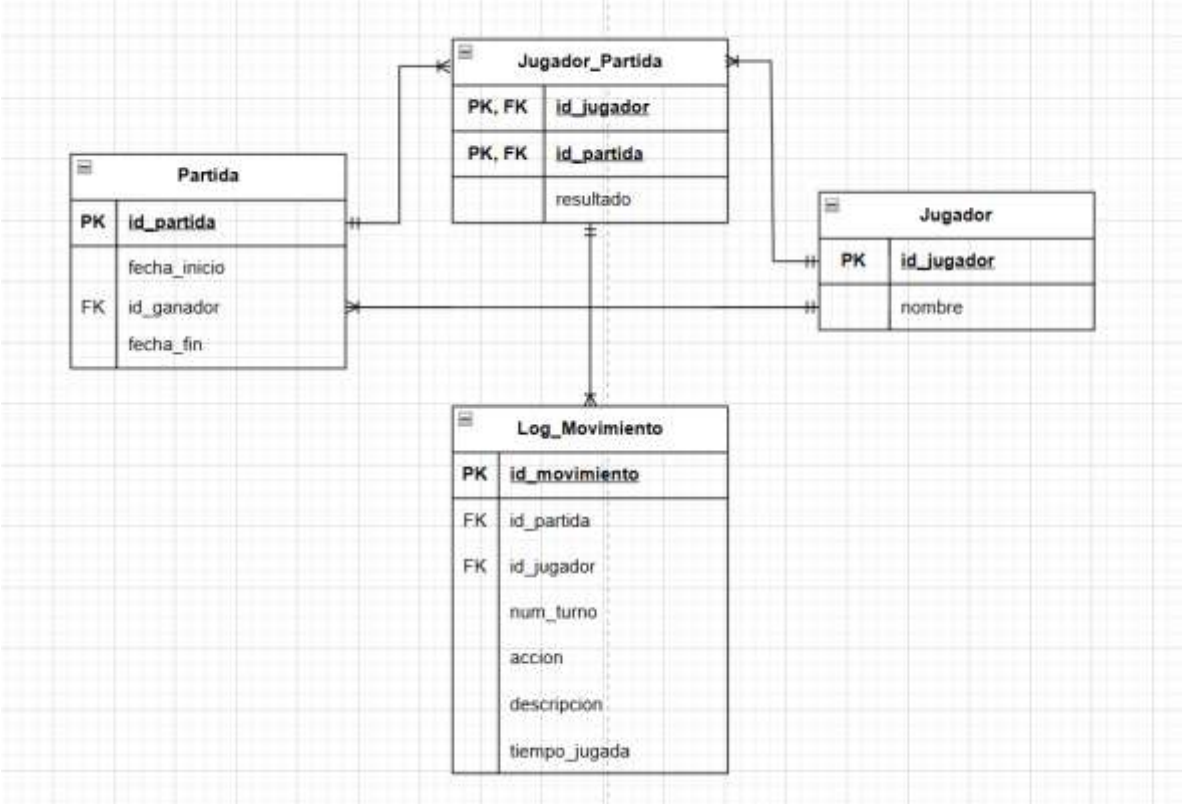
La distribución de tareas siguió su ritmo, pero ahora la parte visual, la modalidad web socket y lógica del juego fue dividida entre un nuevo integrante, Fernando Arredondo, y Alexis Morales.

Distribución de tareas en vista de tabla.

Tarea	Integrante a cargo	Descripción
Base de datos en SQL	Iván Castañón	Creación de la base de datos
DataGridViews		Creación del apartado (incluyendo botones y diseño visual) FormHistorial de partidas.
FormPartida		Rediseño del entorno de la partida e integración de botones para cambio de ventanas.
API y detalles de conexión a la base de datos		Creación de la API e integración al proyecto en VS.
Ajuste y Corrección de Errores		Agregar y/o corregir errores en la implementación de sockets y el 3er jugador
Implementación de API dentro del proyecto en VS		Implementación de todas las funciones y clases de la API (incluyendo formatos JSON) dentro del proyecto en VS. Inicialización de partida y guardado en la base de datos.
Lógica del juego y consideraciones oficiales	Alexis Morales	Lógica del juego (movimientos, tiros, penalizaciones entre otros de acuerdo a las consideraciones oficiales del juego UNO.
Apartados visuales		Diseño y estética visual dentro de cada form (partida e inicio) a excepción de historial.

Clases e implementaciones del juego		Creación de las clases requeridas (Mazo y Carta) junto con sus respectivos métodos y funciones. Creación e implementación de funciones y eventos dentro del juego (tirar cartas, robar cartas, etcétera).
Validación de ensambles con la base de datos y corrección de errores.		Validación de los avances, correcciones necesarias (tanto en el proyecto dentro de VS como en GitHub) e informar sobre errores grandes encontrados.
Redefinido de interfaces en versión final		Validación y detalles finales antes de la entrega.
Implementar la comunicación TCP mediante JSON		Creo y implemento lo necesario los protocolos necesarios para el manejo de sockets y su comunicación entre el servidor y jugadores.
Implementación de Sala	Arredondo Fernando	Añadir y validar los campos y métodos necesarios para una sala de espera.
Actualización de Form1		Se agrega el botón a la pantalla inicial del juego consecuente de la implementación de la sala.
Implementar Jugador 3		Se actualizan los método y variables definidas de 2 jugadores a 3 en todas las clases y métodos necesarios.

Diagrama entidad relación de la base de datos:



En el modelo se puede observar que se crea la tabla Jugador, donde se guardan los datos del jugador, tanto su id, como su nombre.

También creamos la tabla Partida, con su identificador, la fecha y hora en la que se inició esa partida y en qué momento terminó.

A su vez se crea la tabla intermedia de Jugador_Partida, para evitar relaciones de muchos a muchos y simplificarla haciéndolas de uno a muchos, para ello se usan los identificadores de la partida y del jugador, junto con el resultado, el cuál determina si el jugador es ganador o perdedor, y los puntos que completó en la partida, los puntos son la suma de los números de las cartas con las que terminó el jugador en la mano, si el jugador gana entonces los puntos deberían de ser 0.

Por otra parte, también se creó la tabla de Log_Movimiento, en ella se guardan las acciones que se ejecutaron en el turno de cada jugador.

Las relaciones son:

- En una partida puede haber muchos jugadores, un jugador puede jugar muchas partidas.
- Y en una partida cada jugador **puede hacer muchos movimientos.**
- **En una partida puede haber muchos jugadores, pero solo un ganador.**

Lista de Commits en GitHub.

Los commits que cada integrante realice (y ha realizado hasta la fecha de este reporte) integran una pequeña descripción de lo que se realizó por parte del integrante al momento de crear ese commit.

El historial de commits es el siguiente (se muestra el de la rama main porque integra todo el historial de commits de ambas ramas):



Commits on Oct 10, 2026

archivo de requerimientos de la API by Ivan Alexis authored and Alexis committed yesterday	11cf36f
El mazo ahora se rebarajea (toma las del pozo) si se terminan las cartas, implemente sonido de fondo en el juego pero lo elimine por detalles con el hardware y canales de audio de windows Alexis authored and Alexis committed yesterday	f3b18c2
Corregi un error en los indicadores de color a tirar Alexis authored and Alexis committed yesterday	d04a5ff
Agregue indicador de que color debe tirar el jugador si hubo cambio de color Alexis authored and Alexis committed yesterday	0fb43a4
Agregado datagrid Jugador_Partida y correcciones de código Iván authored and Iván committed yesterday	b5a76ad
Agregue indicadores de turno (texto y avatares cambian) Alexis authored and Alexis committed yesterday	b9a38e9
Al comer cartas, se muestran en el inicio del mazo, evitando tener q deslizar en cada tomada de cartas Alexis authored and Alexis committed yesterday	0eaf483
Sin cambios, solo para hacer pull Alexis authored and Alexis committed yesterday	6b7307b
Resuelto problema de jugadores en BD y diseño de historial Iván authored and Iván committed 2 days ago	f13b76d

Commits on Oct 8, 2026

Agregué avatares, corregi nombres de jugadores y alertas en el juego,modificque la interfaz visual del selector de color — 5062286
Alexis authored and Alexis committed 4 days ago

Commits on Oct 7, 2026

Agregué un sonido cuando el jugador gana — 290ca66
Alexis authored and Alexis committed 4 days ago

Agregue sonidos en eventos, una carta que no se mostraba se corrigió y si el pozo comienza con un comodin se cambia a una carta numero — 161f538
Alexis authored and Alexis committed 4 days ago

sin cambios significativos (la base de datos fue modificada pero nooficialmente, mas tarde se corregirá) — 2eed8c3
Alexis authored and Alexis committed 4 days ago

Partidas y movimientos se guardan correctamente — 533cef6
Iván authored and Iván committed 5 days ago

Commits on Oct 6, 2026

Resuelto problema de fusion — 41837fc
Iván authored and Iván committed 5 days ago

Agregué un poco de diseño de interfaz visual, el juego ahora se abr en pantalla completa — 0b5b5bc
Alexis authored and Alexis committed 5 days ago

Api creada y boton ver historial corregido — 6e89670
Iván authored and Iván committed 5 days ago

Commits on Oct 5, 2026

Agregué la ventana para elegir color de cambio en comodines +4 o cambio de color — b8eec2d
Alexis authored and Alexis committed last week

Agregue funcion de castigar jugador, efectos de los comodines, el diseño visual de los mazos (ahora se ven todas las cartas) — 9c2e41f
Alexis authored and Alexis committed last week

Corregi las cartas comodi, ahora todo se muestra en pantalla correctly — 7bae2b5
Alexis authored and Alexis committed last week

Agregue todas las cartas y corregi que no se mostraban los comodines en el maso — 81f79af
Alexis authored and Alexis committed last week

Commits on Oct 4, 2026

Añadi el mazo para comer, y la logica de jugar cartas con el pozo Alexis authored and Alexis committed last week	dcbd97d
Aviso agregado Iván authored and Iván committed last week	c938fc9
Añadi random a las cartas de mazos y deteccion de la carta que se elige Alexis authored and Alexis committed last week	e42626a
El merge esta todo listo y funcionando Alexis authored and Alexis committed last week	4c22b98
Otra vez se coló basura al hacer el merge pero se ha corregido Alexis authored and Alexis committed last week	4a76920
Correción gitignore Iván authored and Iván committed last week	559028d
Commits Ver Historial agregado Iván authored and Iván committed last week	697f3d7
Ya se muestran los masos graficamente, agregué cartas azules y rojas Alexis authored and Alexis committed last week	dab3290
Resuelto conflicto de compilacion obj Iván authored and Iván committed last week	55a1f25
Ya estan reparados los forms, cambié los ImageBox por FlowLayerPanel Alexis authored and Alexis committed last week	03f2644
agregué el gitignore Alexis authored and Alexis committed last week	f1c2c57

Commits on Oct 4, 2026

Todo solucionado con el merge: movi los imageBox a form2 Alexis authored and Alexis committed last week		b4d6213
Merge de Ivan a rama de Alexis (se agregan form2 y 3 Alexis authored and Alexis committed last week		5fd4339
Merge remote-tracking branch 'origin/Ivan_Branch' into Alexis_Branch Alexis authored and Alexis committed last week		43a5204
Ivan implemento otro tipo de ventanas, cambie para adaptarnos a esa implement Alexis authored and Alexis committed last week		8067cd2
Solo imagen para corregir Iván authored and Iván committed last week		559ed27
archivos temporales (nada que ver) Alexis authored and Alexis committed last week		157912b
Update README with course details mxrlxss authored last week	Verified	bd8a8c6
Commit Agregando ventanas Iván authored and Iván committed last week		57ef1a3
Merge remote-tracking branch 'origin/Ivan_Branch' into Alexis_Branch Alexis authored and Alexis committed last week		df9ef61

Commits on Oct 3, 2026

Tablas y relaciones de la Base de Datos creadas de forma exitosa Iván authored and Iván committed last week		2f134fe
Agregue las interfaces (menu inicial y tablero) y algunos botones base Alexis authored and Alexis committed last week		757fc3a
lo mismo del anterior (cartas y mazo) pq no se guardo bien XD Alexis authored and Alexis committed last week		0178d67
Añadi las clases carta y mazo y sus logicas de juego (agregar mazo, robar carta e inicializar tablero Alexis authored and Alexis committed last week		f66fd9f

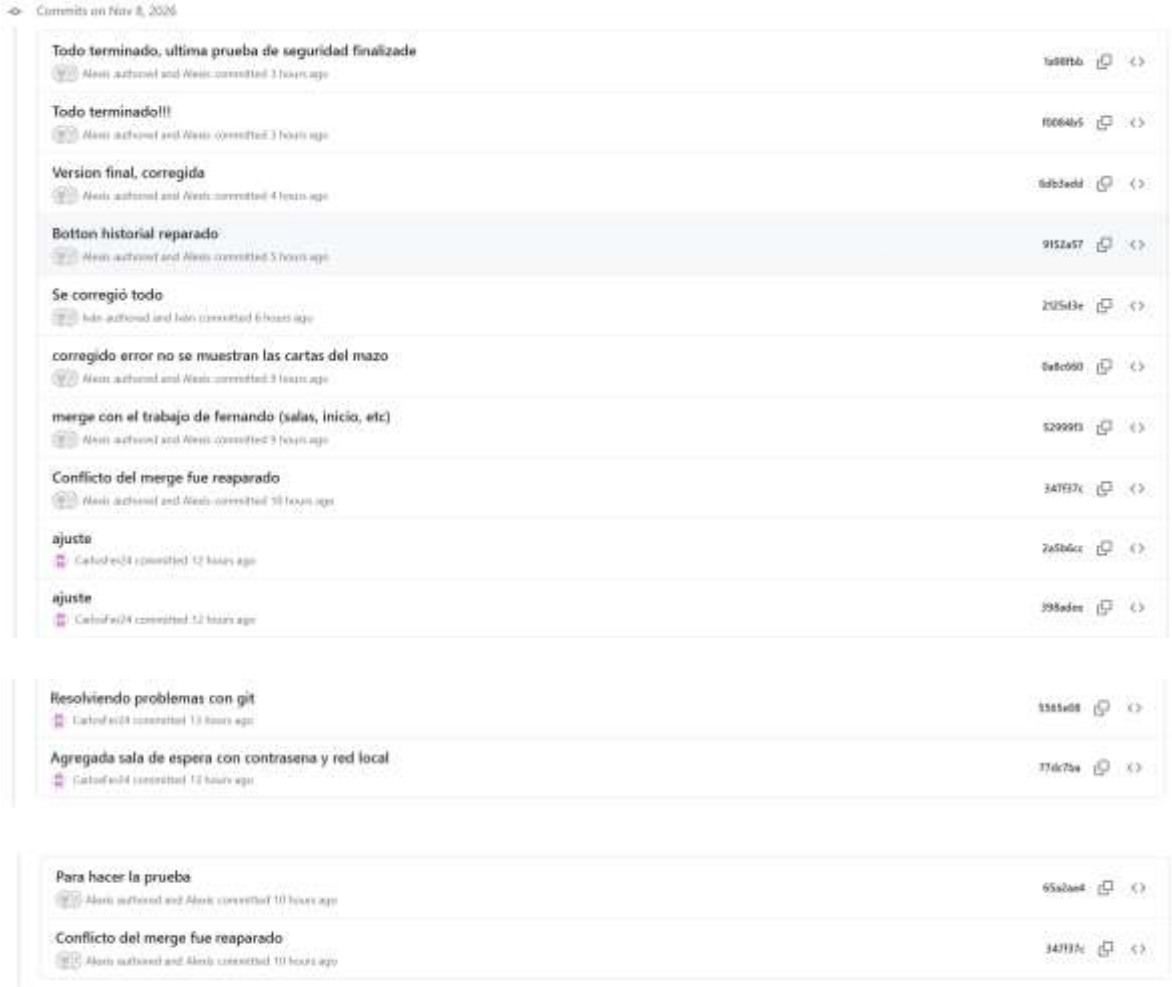
Commits on Oct 2, 2026

Merge remote-tracking branch 'origin/Alexis_Branch' into Ivan_Branch Iván authored and Iván committed last week		d4d6381
archivo de visual Alexis authored and Alexis committed last week		2592b9c
Creacion del proyecto de visual Alexis authored and Alexis committed last week		1d6f368
Agrega el respaldo de la base de datos en MySQL Iván authored and Iván committed last week		ca00609

Commits on Oct 1, 2026

Commit inicial (subiendo carpetas con recursos graficos) Alexis authored and Alexis committed last week		7fa1929
Initial commit marlsss authored last week	Verified	380dada

Segunda entrega de proyecto.



Commits on Nov 7, 2026

Ya va todo bien, los sockets, logica de cartas y turnos, etcétera Alexis authored and Alexis committed yesterday	a7fb958
Funcion escucha de mensajes en la red agregada Alexis authored and Alexis committed yesterday	c619eeb
agregué función manejo de 3 turnos Alexis authored and Alexis committed yesterday	5e1949e
Merge branch 'IvanCasta_Branch' Iván authored and Iván committed yesterday	1dd7bab
Algún error Iván authored and Iván committed yesterday	a259007
API simplificada, clase Dato actualizada en ApiHistorial, Clase FormHistorial fue corregida, todo adaptado a usuarios y servidores Iván authored and Iván committed yesterday	8c64a60
Agregué los sockets y todo lo relacionadoa cliente-serbidor (prueba piloto, AUN NO ESTA BIEN) Alexis authored and Alexis committed yesterday	384d1a3
Agregué bandera del anfitrion Alexis authored and Alexis committed yesterday	4c9166c
Creé form de sala y registro de usuarios Alexis authored and Alexis committed yesterday	5960b5c
Agregados algunos comentarios Iván authored and Iván committed 2 days ago	3809a63
Agregados los Sockets para clientes y servidor Iván authored and Iván committed 2 days ago	f2127fc

Commits on Nov 6, 2026

Cambios en la clase PartidaAPI de C# Iván authored and Iván committed 2 days ago	106e1ff
API actualizada para meter más jugadores a la base de datos Iván authored and Iván committed 2 days ago	fad1aad
Nueva rama creada Iván authored and Iván committed 2 days ago	24f6a78

Ramas de trabajo en GitHub.

Las ramas de trabajo permiten que cada integrante agregue su parte del código sin molestar a otros integrantes, teniendo como ventaja trabajar en entornos aislados para después unir las partes de cada rama en la rama “master” que es la que consideramos como la rama de la entrega final, pues será quien aloje el programa en su versión estable.

Las ramas creadas son las siguientes:

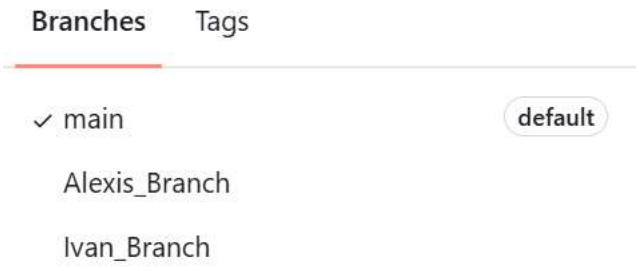
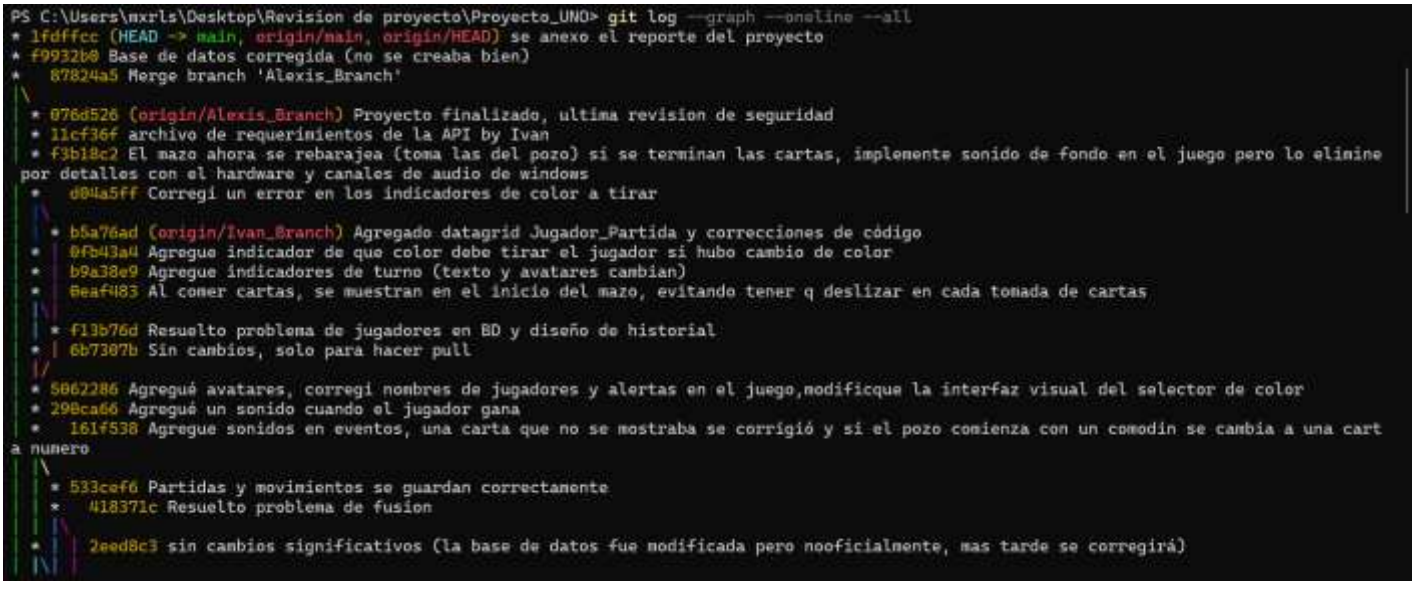


Imagen Ramas:

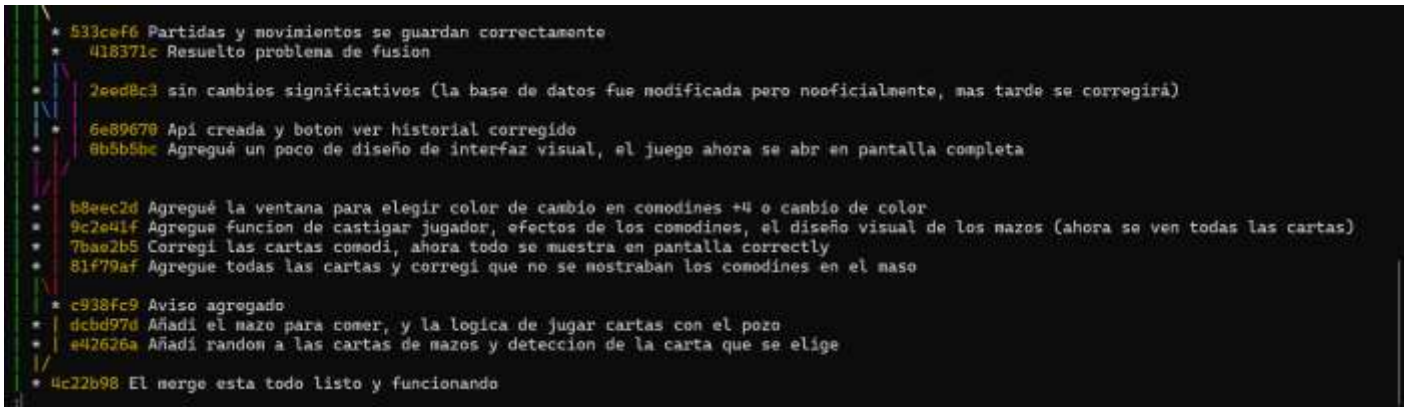


```
* 1fdffcc (HEAD -> main, origin/main, origin/HEAD) se anexo el reporte del proyecto
* f9932b0 Base de datos corregida (no se creaba bien)
*   87824a5 Merge branch 'Alexis_Branch'
|\
| * 076d526 (origin/Alexis_Branch) Proyecto finalizado, ultima revision de seguridad
| * 11cf36f archivo de requerimientos de la API by Ivan
| * f3b18c2 El mazo ahora se rebarajea (toma las del pozo) si se terminan las cartas, implemente sonido de fondo en el juego pero lo elimine
por detalles con el hardware y canales de audio de windows
| *   d04a5ff Corregi un error en los indicadores de color a tirar
| |\
| | * b5a76ad (origin/Ivan_Branch) Agregado datagrid Jugador_Partida y correcciones de código
| * | 0fb43a4 Agregue indicador de que color debe tirar el jugador si hubo cambio de color
| * | b9a38e9 Agregue indicadores de turno (texto y avatares cambian)
| * | 0eaf483 Al comer cartas, se muestran en el inicio del mazo, evitando tener q deslizar en cada tomada de cartas
| |\|
| | * f13b76d Resuelto problema de jugadores en BD y diseño de historial
| * | 6b7307b Sin cambios, solo para hacer pull
| |/
| * 5062286 Agregué avatares, corregi nombres de jugadores y alertas en el juego,modificque la interfaz visual del selector de color
| * 290ca66 Agregué un sonido cuando el jugador gana
| *   161f538 Agregue sonidos en eventos, una carta que no se mostraba se corrigió y si el pozo comienza con un comodin se cambia a una cart
a numero
| |\
| | * 533cef6 Partidas y movimientos se guardan correctamente
| | *   418371c Resuelto problema de fusion
| | |\
| * | | 2eed8c3 sin cambios significativos (la base de datos fue modificada pero nooficialmente, mas tarde se corregirá)
| |\| |
| | * | 6e89670 Api creada y boton ver historial corregido
| * | | 0b5b5bc Agregué un poco de diseño de interfaz visual, el juego ahora se abr en pantalla completa
| |/ /
| |/|
| * | b8eec2d Agregué la ventana para elegir color de cambio en comodines +4 o cambio de color
| * | 9c2e41f Agregue funcion de castigar jugador, efectos de los comodines, el diseño visual de los mazos (ahora se ven todas las cartas)
| * | 7bae2b5 Corregi las cartas comodi, ahora todo se muestra en pantalla correctly
| * | 81f79af Agregue todas las cartas y corregi que no se mostraban los comodines en el maso
| |\|
| | * c938fc9 Aviso agregado
| * | dcbd97d Añadi el mazo para comer, y la logica de jugar cartas con el pozo
| * | e42626a Añadi randon a las cartas de mazos y deteccion de la carta que se elige
| |/
| * 4c22b98 El merge esta todo listo y funcionando
```

```
| |/
| * 4c22b98 El merge esta todo listo y funcionando
| *   4a76920 Otra vez se coló basura al hacer el merge pero se ha corregido
| |\
| | * 559028d Correción gitignore
| | * 697f3d7 Commits  Ver Historial agregado
:
* 1fdffcc (HEAD -> main, origin/main, origin/HEAD) se anexo el reporte del proyecto
* f9932b0 Base de datos corregida (no se creaba bien)
*   87824a5 Merge branch 'Alexis_Branch'
|\
| * 076d526 (origin/Alexis_Branch) Proyecto finalizado, ultima revision de seguridad
| * 11cf36f archivo de requerimientos de la API by Ivan
| * f3b18c2 El mazo ahora se rebarajea (toma las del pozo) si se terminan las cartas, implemente sonido de fondo en el juego pero lo elimine
por detalles con el hardware y canales de audio de windows
| *   d04a5ff Corregi un error en los indicadores de color a tirar
| |\
| | * b5a76ad (origin/Ivan_Branch) Agregado datagrid Jugador_Partida y correcciones de código
| * | 0fb43a4 Agregue indicador de que color debe tirar el jugador si hubo cambio de color
| * | b9a38e9 Agregue indicadores de turno (texto y avatares cambian)
| * | 0eaf483 Al comer cartas, se muestran en el inicio del mazo, evitando tener q deslizar en cada tomada de cartas
| |\|
| | * f13b76d Resuelto problema de jugadores en BD y diseño de historial
| * | 6b7307b Sin cambios, solo para hacer pull
| |/
| * 5062286 Agregué avatares, corregi nombres de jugadores y alertas en el juego,modificque la interfaz visual del selector de color
| * 290ca66 Agregué un sonido cuando el jugador gana
| *   161f538 Agregue sonidos en eventos, una carta que no se mostraba se corrigió y si el pozo comienza con un comodin se cambia a una cart
a numero
| |\
| | * 533cef6 Partidas y movimientos se guardan correctamente
| | *   418371c Resuelto problema de fusion
| | |\
| * | | 2eed8c3 sin cambios significativos (la base de datos fue modificada pero nooficialmente, mas tarde se corregirá)
| |\| |
| | * | 6e89670 Api creada y boton ver historial corregido
| * | | 0b5b5bc Agregué un poco de diseño de interfaz visual, el juego ahora se abr en pantalla completa
| |/ /
| |/|
| * | b8eec2d Agregué la ventana para elegir color de cambio en comodines +4 o cambio de color
| * | 9c2e41f Agregue funcion de castigar jugador, efectos de los comodines, el diseño visual de los mazos (ahora se ven todas las cartas)
| * | 7bae2b5 Corregi las cartas comodi, ahora todo se muestra en pantalla correctly
| * | 81f79af Agregue todas las cartas y corregi que no se mostraban los comodines en el maso
```

```
|\
| * c938fc9 Aviso agregado
* | dcbd97d Añadi el mazo para comer, y la logica de jugar cartas con el pozo
* | e42626a Añadi randon a las cartas de mazos y deteccion de la carta que se elige
|/
* 4c22b98 El merge esta todo listo y funcionando
*   4a76920 Otra vez se coló basura al hacer el merge pero se ha corregido
|\
| * 559028d Correción gitignore
| * 697f3d7 Commits  Ver Historial agregado
| *   55a1f25 Resuelto conflicto de compilacion obj
| |\
| * | 5fd4339 Merge de Ivan a rama de Alexis (se agregan form2 y 3
:
* 1fdffcc (HEAD -> main, origin/main, origin/HEAD) se anexo el reporte del proyecto
* f9932b0 Base de datos corregida (no se creaba bien)
*   87824a5 Merge branch 'Alexis_Branch'
|\
| * 076d526 (origin/Alexis_Branch) Proyecto finalizado, ultima revision de seguridad
| * 11cf36f archivo de requerimientos de la API by Ivan
| * f3b18c2 El mazo ahora se rebarajea (toma las del pozo) si se terminan las cartas, implemente sonido de fondo en el juego pero lo elimine
por detalles con el hardware y canales de audio de windows
| *   d04a5ff Corregi un error en los indicadores de color a tirar
| |\
| | * b5a76ad (origin/Ivan_Branch) Agregado datagrid Jugador_Partida y correcciones de código
| * | 0fb43a4 Agregue indicador de que color debe tirar el jugador si hubo cambio de color
| * | b9a38e9 Agregue indicadores de turno (texto y avatares cambian)
| * | 0eaf483 Al comer cartas, se muestran en el inicio del mazo, evitando tener q deslizar en cada tomada de cartas
| |\|
| | * f13b76d Resuelto problema de jugadores en BD y diseño de historial
| * | 6b7307b Sin cambios, solo para hacer pull
| |/
| * 5062286 Agregué avatares, corregi nombres de jugadores y alertas en el juego,modificque la interfaz visual del selector de color
| * 290ca66 Agregué un sonido cuando el jugador gana
| *   161f538 Agregue sonidos en eventos, una carta que no se mostraba se corrigió y si el pozo comienza con un comodin se cambia a una cart
a numero
| |\
| | * 533cef6 Partidas y movimientos se guardan correctamente
| | *   418371c Resuelto problema de fusion
| | |\
| * | | 2eed8c3 sin cambios significativos (la base de datos fue modificada pero nooficialmente, mas tarde se corregirá)
| |\| |
| | * | 6e89670 Api creada y boton ver historial corregido
```

```
| * | | 0b5b5bc Agregué un poco de diseño de interfaz visual, el juego ahora se abr en pantalla completa
| | |/
| |/|
| * | b8eec2d Agregué la ventana para elegir color de cambio en comodines +4 o cambio de color
| * | 9c2e41f Agregue funcion de castigar jugador, efectos de los comodines, el diseño visual de los mazos (ahora se ven todas las cartas)
| * | 7bae2b5 Corregi las cartas comodi, ahora todo se muestra en pantalla correctly
| * | 81f79af Agregue todas las cartas y corregi que no se mostraban los comodines en el maso
| |\
| | * c938fc9 Aviso agregado
| * | dcbd97d Añadi el mazo para comer, y la logica de jugar cartas con el pozo
| * | e42626a Añadi randon a las cartas de mazos y deteccion de la carta que se elige
| |/
| * 4c22b98 El merge esta todo listo y funcionando
| *   4a76920 Otra vez se coló basura al hacer el merge pero se ha corregido
| |\
| | * 559028d Correción gitignore
| | * 697f3d7 Commits  Ver Historial agregado
| | *   55a1f25 Resuelto conflicto de compilacion obj
| | |\
| | * | 5fd4339 Merge de Ivan a rama de Alexis (se agregan form2 y 3
:
* 1fdffcc (HEAD -> main, origin/main, origin/HEAD) se anexo el reporte del proyecto
* f9932b0 Base de datos corregida (no se creaba bien)
*   87824a5 Merge branch 'Alexis_Branch'
|\
| * 076d526 (origin/Alexis_Branch) Proyecto finalizado, ultima revision de seguridad
| * 11cf36f archivo de requerimientos de la API by Ivan
| * f3b18c2 El mazo ahora se rebarajea (toma las del pozo) si se terminan las cartas, implemente sonido de fondo en el juego pero lo elimine
por detalles con el hardware y canales de audio de windows
| *   d04a5ff Corregi un error en los indicadores de color a tirar
| |\
| | * b5a76ad (origin/Ivan_Branch) Agregado datagrid Jugador_Partida y correcciones de código
| * | 0fb43a4 Agregue indicador de que color debe tirar el jugador si hubo cambio de color
| * | b9a38e9 Agregue indicadores de turno (texto y avatares cambian)
| * | 0eaf483 Al comer cartas, se muestran en el inicio del mazo, evitando tener q deslizar en cada tomada de cartas
| |\|
| | * f13b76d Resuelto problema de jugadores en BD y diseño de historial
| * | 6b7307b Sin cambios, solo para hacer pull
| |/
| * 5062286 Agregué avatares, corregi nombres de jugadores y alertas en el juego,modificque la interfaz visual del selector de color
| * 290ca66 Agregué un sonido cuando el jugador gana
| *   161f538 Agregue sonidos en eventos, una carta que no se mostraba se corrigió y si el pozo comienza con un comodin se cambia a una cart
a numero
```



Segunda entrega.

Se agregó un nuevo integrante al equipo, Fernando, para el cual se creó una rama de trabajo *Fernando_Branch.*

```
PS C:\Users\mxrls\Desktop\Revisio\Proyecto_UNO> git log --graph --oneline --all
* 1a98fbb (HEAD -> Alexis_Branch, origin/Alexis_Branch) Todo terminado, ultima prueba de seguridad fina
lizade
* f0084b5 Todo terminado!!!
* 6db3add Version final, corregida
* 9152a57 Botton historial reparado
* 2125d3e Se corregió todo
* 0a8c660 corregido error no se muestran las cartas del mazo
*   52999f3 merge con el trabajo de fernando (salas, inicio, etc)
```

```
* 2a5b6cc (origin/Fernando_Branch) ajuste
* 398adee ajuste
*   5565e08 Resolviendo problemas con git
* | 77dc7ba Agregada sala de espera con contrasena y red local
* 65a2aa4 (origin/main, origin/HEAD, main) Para hacer la prueba
*   347f37c Conflicto del merge fue reaparado
*   1dd7bab Merge branch 'IvanCasta_Branch'
* | e239007 Algún error
*   e7fb958 Ya va todo bien, los sockets, logica de cartas y turnos, etcétera
* 8c64a60 (origin/IvanCasta_Branch) API simplificada, clase Dato actualizada en ApiHistorial, Clase
FormHistorial fue corregida, todo adaptado a usuarios y servidores
* 3809e63 Agregados algunos comentarios
* f2127fc Agregados los Sockets para clientes y servidor
* f06e1f1 Cambios en la clase PartidaAPI de C#
* fad1aad API actualizada para meter más jugadores a la base de datos
* 24fda78 Nueva rama creada
```

Descripción de la API.

Nuestra API cuenta con 6 Endpoints:

Get /partidas: Devuelve toda la información relacionada a la partida del juego

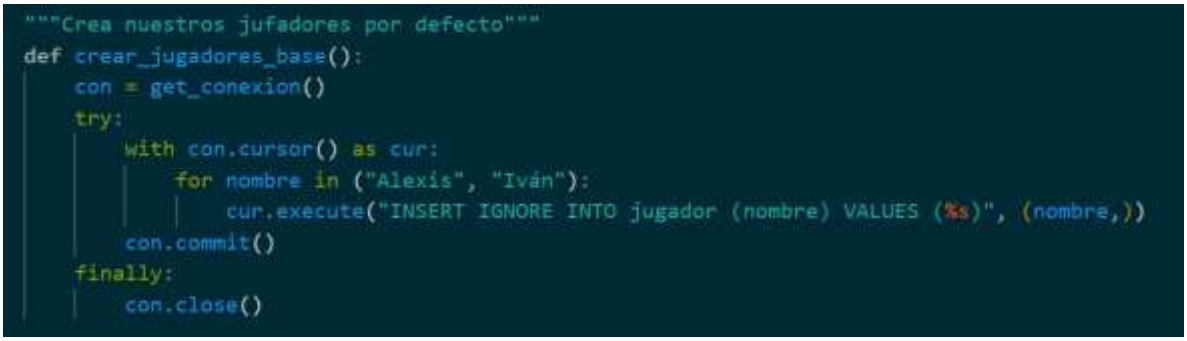
Get /partidas/{idPartida/jugadores: Devuelve la información relacionada a los jugadores que jugaron en esa partida y si son ganadores o perdedores.

Get /partidas/{idPartida}/movimientos: Devuelve la información relacionada a los movimientos de esa partida.

Post /partidas: Crea una nueva partida al momento de presionar el botón “Iniciar partida en el juego”.

Post /partidas/{idPartida}/movimientos: Crea directamente el movimiento hecho en ese turno por el jugador.

En este caso aún no existe un post que cree a los jugadores, para poder introducir a los únicos dos jugadores disponibles usamos una función, porque solo somos dos jugadores, no queríamos complicarnos mucho, he aquí la función:



```
"""Crea nuestros jufadores por defecto"""
def crear_jugadores_base():
    con = get_conexion()
    try:
        with con.cursor() as cur:
            for nombre in ("Alexis", "Iván"):
                cur.execute("INSERT IGNORE INTO jugador (nombre) VALUES (%s)", (nombre,))
        con.commit()
    finally:
        con.close()
```

Estamos conscientes de que en el siguiente parcial necesitaremos crear un post para crear jugadores de cada partida.

Put /partidas/{idPartida}/finalizar: Aquí se actualizan los valores de cada partida que quedaron pendientes, como fecha final y si un jugador ganó en la partida entonces pone el nombre.

Problemas en el camino.

Problemas con la configuración de GitHub

Al iniciar a trabajar con GitHub, creamos las ramas de cada integrante y uno de nosotros realizo el proyecto de Visual Studio para después compartirlo y que todos pudiéramos trabajar. El problema comenzó cuando iniciamos a intercambiar avances con Git merge, pues arrojaba errores de auto-merge y cosas por el estilo. Fue muy tedioso estar revisando error por error, al punto en el que consideramos trabajar con respaldos locales de forma manual (copiábamos cada avanza a una carpeta temporal en nuestras computadoras por si algo pasaba, a pesar de que GitHub tiene el control de versiones pues solo lo hacíamos por mera precaución y desde nuestra poca experiencia en este entorno de trabajo).

Después de muchas búsquedas e indagaciones en internet y en agentes de IA, pudimos resolver la falla.

Primera falla: Del lado de Alexis, su carpeta de trabajo estaba en una carpeta de OneDrive, lo cual generaba problemas debido a que en todo momento se estaba haciendo la copia de seguridad en la nube y esto interfería con GitHub, por lo que Alexis cambió a una carpeta fuera de la nube. Con esto se resolvió una buena parte del problema.

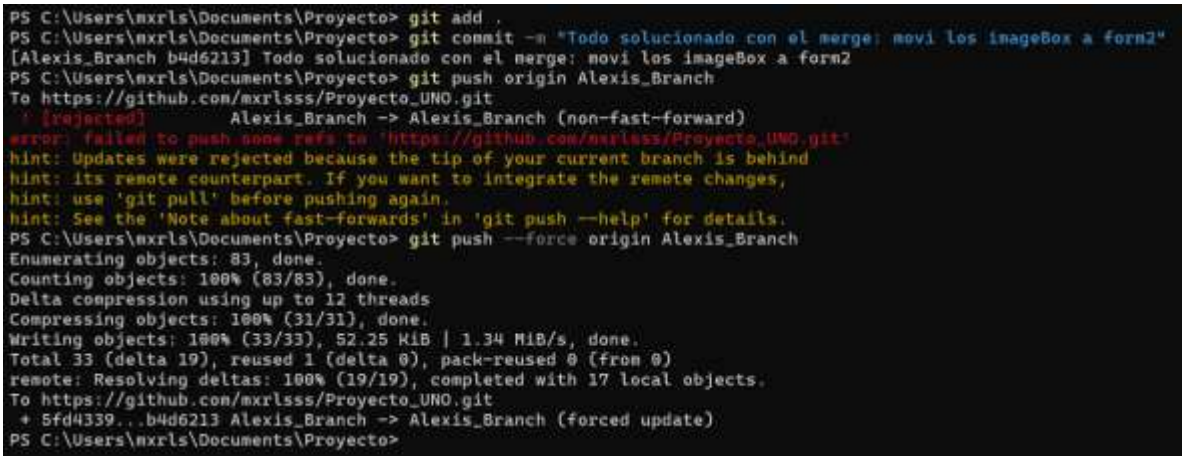
Segunda falla: Por un error de escritura, el archivo *gitignore (el cual tenía la función de ignorar los archivos basura que Visual Studio genera en cada compilación del proyecto)* tenía extensión .txt por lo que no era reconocido por Git y entonces toda la basura se subía y a la hora de realizar merge, tronaba todo. Una vez que el archivo fue corregido, solucionamos por completo la falla del merge.

Después de que pudimos hacer merge correctamente, todo estaba listo para continuar y a partir de ahí, únicamente fue realizar pull (**git pull origin *rama del compañero*)** cada vez que comenzábamos a trabajar para descargar y unir los

cambios que el compañero había realizado en su rama, evitándonos hacer merge cada vez que comenzáramos a trabajar.

GitHub desconoció el push después del retroceso temporal en commits

Al querer subir los cambios, como retrocedimos (por lo mencionado anteriormente) git no reconocía esos commits nuevos y decía que lo actualizáramos para que todo coincidiera (pero buscando una solución en internet y recurriendo a la IA, Gemini explícitamente prohibió hacer eso porque nos iba a tirar toda la corrección que acabábamos de hacer y fallaría todo de nuevo) por lo que forcé el push.

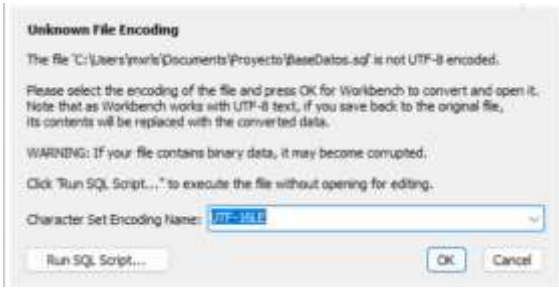


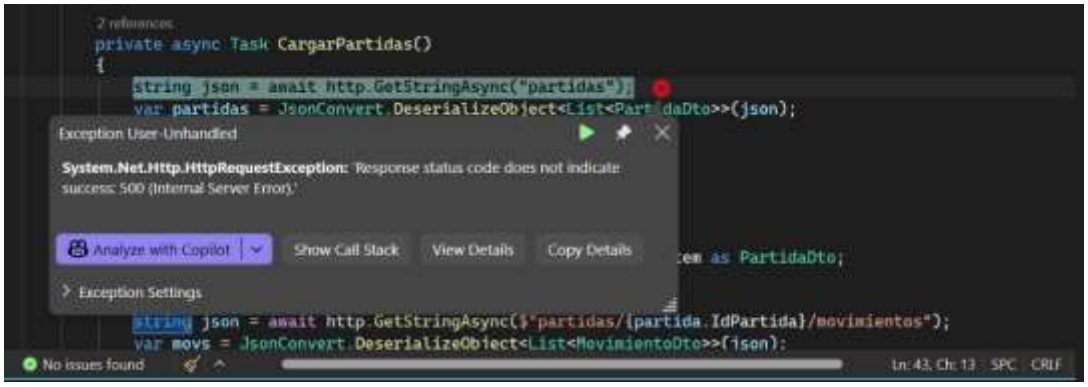
Como GitHub todavía tiene guardado ese historial "malo" en la nube, se asusta al ver que tu computadora tiene un historial completamente distinto y más corto, por lo que te sugiere hacer un `git pull` para igualarlos. **¡Ni se te ocurra ejecutar el `git pull` !** Si lo haces, volverás a descargar toda la basura que acabamos de limpiar y se volverá a romper el proyecto.

Como tú tienes la versión correcta y funcional en tu máquina, tenemos que obligar a GitHub a borrar su historial defectuoso y aceptar tu versión local como la única verdad.

La base de datos no era reconocida por el servidor de uvicorn

Al ejecutar por primera vez en mi computadora (Computadora de Alexis) el servidor en Uvicorn funcionaba normal, pero al hacer las requests a la base de datos tiraba un error desconocido. Al inicio me pareció un poco extraño el formato que mi compañero entregó (MySQL Wb lo reconoció como formato UTF-16) y al abrirlo el estilo era un poco distinto al tradicional de SQL. Después le pedí que revisáramos eso y me entrego en formato de texto plano las sentencias y las ejecuté, pero el error aun seguía (a este punto ya sabíamos que el problema era la base de datos y no algo relacionado con la API o el servidor Uvicorn porque el programa tronaba en la parte en la que se hacía el request al servidor a través de la API).





Apoyados del agente de IA Gemini (porque realizando nuestra revisión e incluso preguntando al profesor no pudimos encontrar el problema, porque no estábamos mirando al lugar correcto y los detalles de error en uvicorn no daban mucho de que hablar) encontramos que tampoco (al inicio) podía ver que la falla estaba en la construcción de la base de datos:



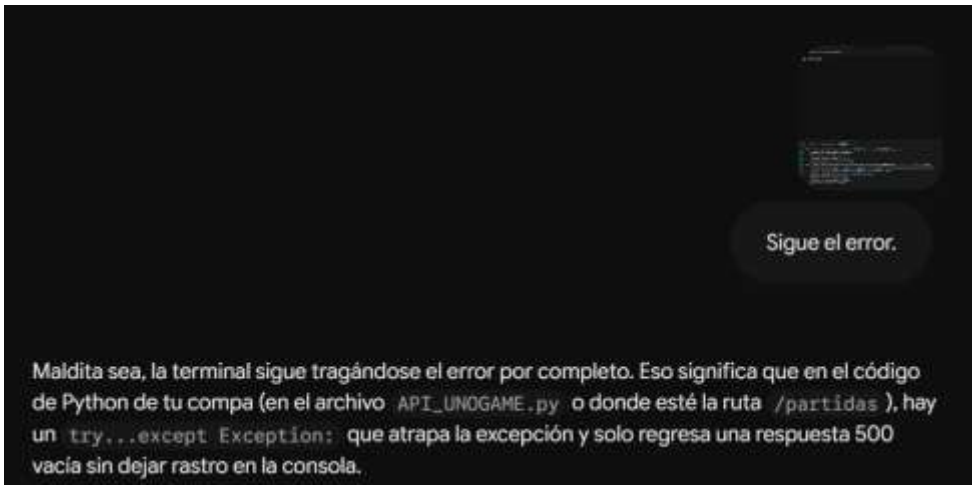
En este punto, el agente estaba seguro de que la base de datos cortaba la conexión y sugirió lo de las credenciales como se indica adelante.

Al inicio el error identificado fue que las credenciales de la base de datos no coincidían porque no establecimos una credencial global.

> “raise errorclass(errno, errval, sqlstate=sqlstate)
> pymysql.err.OperationalError: (1045, "Access denied for user 'root'@'localhost' (using password: YES)")”
> *- da este error el servidor*

Una vez que solucionamos eso, seguía dando error entre las requests al servidor, pero para esto ya habíamos descartado la API, el servidor y el proyecto en VS, por lo que el único problema era la conexión del servidor a la base de datos.

A este punto las cosas estaban un poco tediosas, porque ninguna modificación ni los detalles de error de uvicorn nos daban un horizonte e incluso el agente de IA no sabía ni cual podía ser la falla a tal punto de que lanzó una respuesta un tanto extraña:

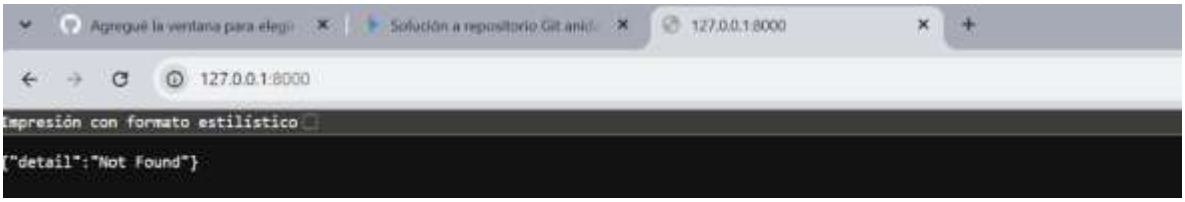


Mencionaba una excepción la cual no existía.

Indagando profundamente descubrimos que (por un error de descuido al no verificar el .SQL antes de ejecutarlo en mi maquina) el script de mi compañero no tenía la creación de la base de datos ni las tablas, por lo que obviamente mi computadora ejecutaba las sentencias de la API sin tener algo en que trabajar (la base de datos).

Posteriormente, modificamos el script de SQL agregando lo requerido y volvimos a correr la prueba, lo cual resultó exitoso.

> *Es importante mencionar que la validación del script de SQL pasó por muchos detalles que ni el agente de IA pudo encontrar (ya que los detalles de error en uvicorn no daban mucha información) y en ese punto no sabíamos si la API tenía errores, si el proyecto en Visual Studio tenia problemas o si la base de datos era el problema, es decir, fuimos descartando cada uno de los posibles escenarios de problemas.*



Finalmente, modificando el script de SQL el error fue solucionado, y ahora la base de datos ya se generaba correctamente desde cero y todo el proyecto funcionaba bien.

Manejo de cadenas JSON

Para poder manejar cadenas JSON primero se tenía que descargar el paquete *Mycrosoft.Json* que permitía su lectura y su creación, con ello venían algunas funciones muy utilizadas como:

```
JObject.Parse.
```

```
await http.PutAsync($"partidas/{idPartida}/finalizar",
Json(new { id_ganador = IdJugador(jugadorGanador) }));
```

Ya con ello ahora sí pude trabajar correctamente con JSON.

Jugadores revueltos en la base de datos.

Otro problema que afectó bastante fue cuando iniciábamos otra partida, al momento de terminarla y un jugador ganaba, se guardaba como ganador el otro jugador. Tuve que modificar el atributo del nombre del jugador para volverlo UNIQUE. Pero también tuve que eliminar todos los datos guardados en todas las tablas para que pudiese funcionar.

```
SET FOREIGN_KEY_CHECKS = 0;
TRUNCATE TABLE log_movimiento;
TRUNCATE TABLE jugador_partida;
TRUNCATE TABLE partida;
TRUNCATE TABLE jugador;
SET FOREIGN_KEY_CHECKS = 1;

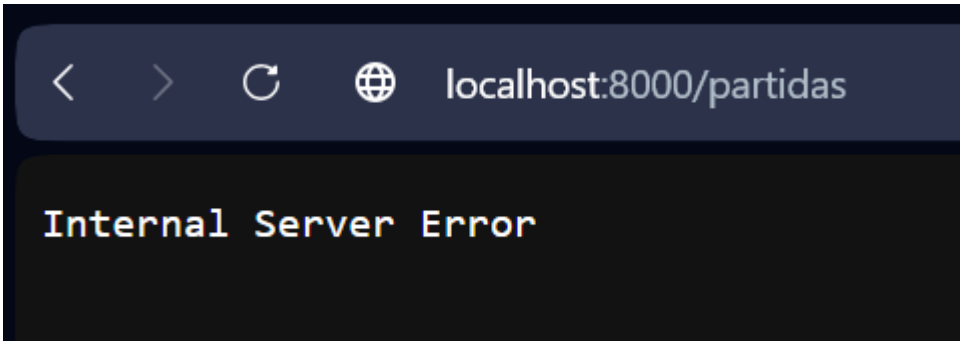
ALTER TABLE jugador ADD UNIQUE (nombre);
```

Problemas con la API y el servidor.

También tuve bastantes problemas con la API en Python, me arrojaba muchos resultados en el navegador cuando trataba de verificar que estuviera mandando todo bien a la URL, a veces de plano no funcionaba y salían errores como este:



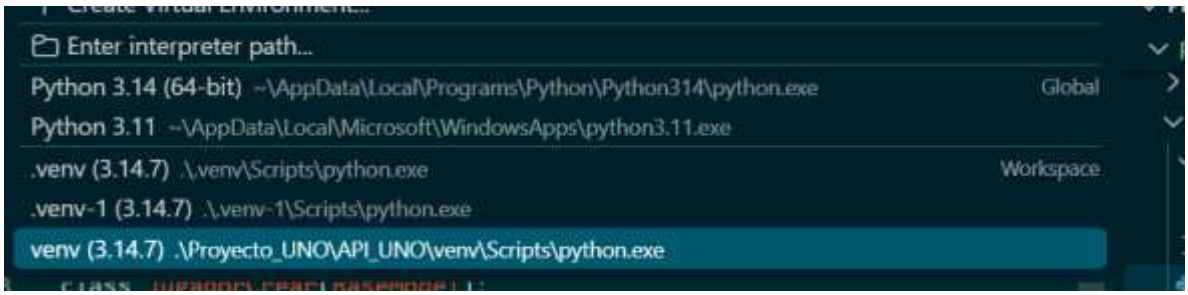
Y el más horrible era este:



Porque no te decía ni donde estaba el error, ni en qué línea, básicamente tenía que leer línea por línea y estar atento en todo momento a la URL para ver si no pasaba.

Problemas con entorno virtual Venv

Además, hubo problemas con el entorno virtual ***.venv,*** porque directamente creaba varios en muchas ubicaciones innecesarias en la carpeta del juego, y luego tenía que decirle a Python que se enfocara solamente en el de la API, me tardé mucho porque estaba bien confundido:



Segunda entrega de proyecto.

Problemas con el dibujado de los mazos.

Una vez que teníamos el código listo, con las adaptaciones requeridas para los sockets, todo funcionaba bien al iniciar una partida, pero la carta del pozo y los mazos de cada jugador no se mostraban.

La carta del pozo tenía un problema con el mensaje enviado a los invitados, pues se actualizaba como una "tirada" y por ello cada jugador tenía un pozo distinto. Fue corregido cambiando la forma en la que escuchaban los clientes y la manera en que el servidor enviaba los datos.

A raíz de esto, cambiaron muchas cosas, porque no estábamos teniendo conexión entre las máquinas en lo referente a movimientos y actualización, porque respecto a la conexión o que las maquinas "se vieran en la red" no había problema alguno.

Después de tantos cambios y modificaciones, agregamos un tag de información en la parte superior de la ventana que nos indicaría el numero de jugador, turno actual y otra información respecto a la pantalla y el dibujado. Con esta información nos dimos cuenta de que todo estaba bien y debía funcionar, pero por alguna razón los mazos de los jugadores seguían sin dibujarse.

Entre tanas búsquedas dimos con el clavo, las coordenadas de dibujado del mazo (*FlowLayoutPanelP2)* estaban en coordenadas fuera de la pantalla, entonces si se dibujaban, pero en una parte que ya no se veía.

Una vez corregido, todo funcionó como lo esperado.

Problemas con la base de datos en un invitado.

Cuando abríamos el historial de partidas en una computadora que no era el servidor, es decir en un invitado, no se mostraba nada y saltaba la excepción de fallo de “timeout”. Lógicamente todo debía funcionar, el código no tenía problemas. Probando la URL en el navegador nos dimos cuenta de que la base de datos no estaba en línea o no era accesible, probamos desactivando el firewall de Windows y resulto ser ese el problema. Con el firewall desactivado la URL en el navegador si cargaba las partidas y por consecuente dentro del juego ya funcionaba esa parte.